AgentCity — Project Specification & System Architecture

🏗️ Version 2 — Product and technical design specification. AgentCity is a concept under design; no complete implementation is represented as working. “Target,” “must” and “done when” describe requirements. Proposed policies and unresolved decisions are explicitly labeled. This revision expands the existing document without changing the core calculator-first scope.

1. Abstract and product definition
    1.1 Abstract
    1.2 Product promise
    1.3 Users and roles
    1.4 Scope boundaries
2. Core objectives and success criteria
3. What the finished experience will look like
    3.1 The world
    3.2 The three product surfaces
    3.3 Example: job agreement card
    3.4 Example: purchase and construction
    3.5 Example: inspection and completion
    3.6 Player-experience boundaries
4. Shared definitions, data ownership and system rules
    4.1 Domain glossary
    4.2 Sources of truth

4.3 Non-negotiable invariants
5. Architecture and dependency model
5.1 Layers
5.2 Principal dependency paths
6. Module specifications
6.1 Game, identity and access
M01 — World server and Paper plugin
M02 — Player interface and account linking
6.2 Jobs, workflow and discovery
M03 — Agreement and team domain
M04 — Workflow orchestrator
M05 — Arkiv marketplace adapter
6.3 Decision model and physical agents
M06 — 0G inference adapter and decision model
M07 — Tool policy and execution authorization
M08 — Blueprint registry and building agent
6.4 Stock, tokens, contracts and procurement
M09 — Warehouse and backing ledger
M10 — CRED spending-token contract
M11 — KIT voucher and issuance contract
M12 — Job escrow and contributor entitlements
M13 — Procurement adapter and Uniswap v4 market
6.5 Translating game conditions into market rules
M14 — State publisher and GameStateRegistry
M15 — GameStateMarketHook
6.6 Delivery, verification and operations
M16 — Redemption claim and delivery worker
M17 — Functional checker and inspection attestor

M18 — Reports, progress and completion history

M19 — Recovery and administration

7. Data and interface contracts

7.1 Core records

7.2 Cross-module command envelope

7.3 Event and transaction handling

7.4 Dependency availability

8. End-to-end operating flows

8.1 Flow A — Request, discovery and agreement

8.2 Flow B — Backing, trade and delivery

8.3 Flow C — Model decision to real construction

8.4 Flow D — Inspection, repair and acceptance

8.5 Flow E — Game-state change to hook enforcement

8.6 Flow F — Teams and departures

9. Lifecycle, money and failure semantics

9.1 Job transitions

9.2 Budget and custody

9.3 Deadlines and cancellation

9.4 Failure-response matrix

10. Delivery plan and definition of done

10.1 Dependency-led build order

10.2 Acceptance checklist for the finished MVP

10.3 Decisions still required

10.4 What this specification does and does not establish

11. Technical basis and references

1. Abstract and product definition

1.1 Abstract

AgentCity is a community build guild and resource marketplace inside a Minecraft server. A player can request a useful build, choose another player or team to deliver it, obtain the required materials, watch a controlled agent construct it, inspect the result and accept the work before the labor reward is released.

The first supported commission is a redstone calculator. Its two inputs each accept values from 0 to 3, and its output displays their sum. The requester does not need to understand redstone construction. They agree to a clear job, continue playing while work happens and receive a calculator whose behavior can be tested against all 16 input combinations.

Arkiv supports shared discovery and marketplace records. A model served through 0G Compute interprets supported requests and proposes bounded actions. Uniswap v4 exchanges standardized material vouchers, with a custom hook that applies trading rules based on game conditions reported onchain.

AgentCity itself must build the server plugin, job workflow, agent controller, inventory custody, token issuance, redemption, functional inspection and settlement contracts.

1.2 Product promise

“Ask for something useful, agree on the work, and keep playing while a builder and their agent complete it.”

The core product is the complete commission, not any individual protocol integration. A successful session ends with a working structure in the world, consumed materials, a recorded inspection and an accepted payment.

1.3 Users and roles

Role	Definition	Primary authority
Requester	Player commissioning the build	Select offer, approve scope and spending, accept finished work
Builder	Player responsible for delivering the agreed build	Offer services, operate an agent, construct and repair

Role	Definition	Primary authority
Team lead	Contributor coordinating a team job	Assign approved tasks and submit the shared result; cannot change others' pay
Contributor	Approved team member with a defined responsibility and reward	Work within assigned permissions; claim their allocation after acceptance
Agent operator	Player controlling a registered bot	Authorize the bot's bounded tools; accountable for its game actions
Supplier	Party depositing material bundles for voucher issuance	Deposit stock, receive authorized KIT, trade it
Server operator	Party running the world and trusted services	Maintain the world, inventory, checker, publishing services and recovery
Automated agent	Software acting for an authorized operator	Execute permitted tasks; no independent authority to approve payment

A person can hold several roles, but the demonstration must include at least two independently controlled participants. Shared supplier/operator roles must be disclosed.

1.4 Scope boundaries

Area	First complete MVP	Team-ready design / later extension
World	One server, one compatible version, one job plot	Multiple protected plots after isolation is proven
Builds	One fixed calculator blueprint	Additional versioned templates
Participants	One requester, one paid builder, one bot	Multiple contributors with fixed allocations
Materials	One defined KIT bundle	Additional separately defined resource assets
Market	One CRED/KIT v4 pool with one state-aware hook	More markets only after inventory reconciliation works

Area	First complete MVP	Team-ready design / later extension
Settlement	One chosen EVM settlement chain	Cross-chain settlement is excluded
Agent autonomy	Supported task selection, execution and bounded repair	No arbitrary redstone invention
Acceptance	Deterministic tests plus explicit requester acceptance	Dispute resolution is a separate future design

ENS, an autonomous city, unrestricted aesthetic judgment and autonomous dispute resolution are outside the initial scope.

2. Core objectives and success criteria

ID	Objective	Observable success criterion
G1	Make commissioning understandable	A player can review scope, controls, budget, roles and acceptance rules before signing
G2	Enable independent marketplace participation	Requester and builder publish/discover the job and offer through Arkiv
G3	Establish shared expectations	Every approval refers to the same immutable agreement version
G4	Make purchased resources necessary	A real swap leads to redemption, delivery and consumption in construction
G5	Give the decision model meaningful work	A recorded 0G decision selects a permitted construction or repair action
G6	Connect reported game state to trading	Healthy purchase succeeds; unavailable/stale purchase is rejected by our pool
G7	Verify actual functionality	Correct calculator passes 16/16, carry fault fails six cases, repair restores 16/16
G8	Settle accepted work once	Only authorized acceptance enables the fixed contributor rewards
G9	Recover across system boundaries	Retries/restarts do not duplicate issuance, delivery, purchase or payout

ID	Objective	Observable success criterion
G10	Make contribution history useful	A completed project links its agreement, report and settlement evidence

Definition of the end result: a usable Minecraft experience with a small wallet companion, rather than a collection of disconnected protocol demonstrations. Technical events should become clear player-facing states.

3. What the finished experience will look like

The following is a target interface specification, not a description of an implemented interface.

3.1 The world

The player enters a shared building hub with a job board, a builder directory, a materials counter and an entrance to the active project plot. These can be simple physical stations backed by menus; decorative city-building is not required for the MVP.

At the plot, the player sees the bot moving, obtaining items and placing blocks. A sign or status panel identifies the job and its stage. Other players can observe, but editing permission is controlled.

The finished calculator has four labeled input levers: A1, A2, B1 and B2, with weights 1 and 2 per input. Three output lamps are labeled 1, 2 and 4. The controls are reachable, readable and match the approved manifest.

3.2 The three product surfaces

Surface	What belongs here	What should stay elsewhere
In-game world and menus	Browse jobs, inspect builders, choose a supported request, follow construction, submit and view tests	Private keys, raw model prompts and protocol administration
Wallet companion	Link identity, review agreement, authorize spending, sign transactions,	Routine block-by-block construction controls

Surface	What belongs here	What should stay elsewhere
	inspect receipts and claim rewards	
Operator console	Health, queue failures, inventory reconciliation, state publication, incident recovery	Requester acceptance or arbitrary alteration of funded rewards

Protocol names can appear in receipt details. The primary buttons should say what they do for the player.

3.3 Example: job agreement card

Field	What the player sees
Job	Calculator #42
Build	Add A and B, each from 0 to 3
Location	Workshop Plot 1
Controls	Four input levers, three output lamps
Verification	All 16 combinations must pass
Materials	One Calculator KIT; spend at most 3 CRED
Builder reward	6 CRED held until acceptance
Team	Builder name and registered bot; additional members if enabled
Timing	Exact agreed work/review deadlines
Changes	Scope and rewards require a new approved version
Actions	Approve agreement / Return for changes

The card must also expose cancellation and timeout treatment before approval. Gas and inference costs appear separately from materials and labor.

3.4 Example: purchase and construction

Stage	Visible content	Available action
Quote ready	“One material kit: 2.7 CRED. Maximum authorized: 3 CRED.”	Review purchase
Service unavailable	“Material delivery is unavailable. New purchases are paused.”	View status / Retry later

Stage	Visible content	Available action
Claim confirmed	“Your materials are reserved. Delivery is pending.”	View delivery claim
Building	“Circuit assembly in progress. Casing complete.”	View job / Visit plot
Blocked	“The builder needs an additional approved resource.”	Review request; no automatic extra charge

The displayed reason must match the underlying operation. “Paid” must never appear solely because a transaction was submitted.

3.5 Example: inspection and completion

Inspection field	Example
Build revision	Revision 7
Result	10/16 passed — repair required
Failure example	A=3, B=1; expected 4, observed 0
Explanation	High carry output is missing; inspect carry path
Labor status	Held; acceptance unavailable
Actions	View failed cases / Return to repair

After repair, the report changes to 16/16 passed. The structural plot remains protected while the requester tries the levers and reviews the work.

The completion card shows: accepted revision, contributor allocations, payment/claim status, full report and previous-work reference. In a team job, each contributor sees their own allocation and claim status.

3.6 Player-experience boundaries

A passing test means ready for review, not automatic payment. A trade means KIT acquired, not materials delivered. A burned KIT means a delivery claim exists, not that the chest has already been filled.

The UI must explain these distinctions through status and available actions. Technical recovery should not require the player to submit the same purchase again.

4. Shared definitions, data ownership and system rules

4.1 Domain glossary

Term	Precise meaning
Job	The commission identified by a stable job ID
Offer	A builder's proposed service, price and availability; not yet an agreement
Agreement	Approved, versioned terms binding participants, scope, budgets, deadlines and acceptance
Blueprint	Versioned placement and construction instructions for a supported build
Test manifest	Versioned interface coordinates, input cases, reset procedure and settling rules
Plot	Protected game region assigned to one active job
Build revision	Server-tracked structural version of the plot; changes invalidate earlier readiness
Task	Bounded unit of work with an assigned actor, preconditions and result
Agent decision	Model proposal for a next action; not execution authority
CRED	Proposed demonstration spending token
KIT	Fungible voucher representing a defined calculator-material bundle
Deposit	Unique supplier event that locks stock before issuance
Redemption claim	Durable obligation created when KIT is burned for a job delivery
Inspection report	Recorded functional results for an agreement and build revision
Acceptance	Requester's explicit authorization to settle the inspected result
State attestation	Authorized report of service conditions; not independent proof of game reality
Hook	Pool-attached contract that applies rules at a v4 operation callback
Reconciliation	Comparing recorded obligations with chain/game evidence before retrying uncertain work

4.2 Sources of truth

Domain	Owner	Other systems' role
Blocks, item custody and structural revision	Paper plugin/world storage	Backend reads observations; model proposes actions
Jobs, task leases and operation history	Workflow database	Arkiv exposes searchable projections
Published profiles and independent offers	Arkiv records with ownership/signature checks	Backend validates and caches
Approved funded agreement	Contract digest plus retained signed agreement	Arkiv provides discovery/reference
Token balances, claims and reward entitlements	Settlement-chain contracts	UI/listeners derive confirmed views
Reported availability	GameStateRegistry	Publisher reports; hook consumes
Functional findings	Deterministic checker over locked game state	Model interprets failures; report storage retains evidence

There is no single database transaction spanning Minecraft, Arkiv, 0G and settlement. Each crossing needs a recorded request, evidence of completion and a recovery rule.

4.3 Non-negotiable invariants

1. The bot builds only inside its authorized plot and consumes delivered materials.
2. No approved budget, recipient or reward allocation changes silently.
3. A deposit identifier authorizes issuance at most once.
4. Reserved stock covers outstanding KIT plus burned-but-undelivered claims, item by item.
5. A redemption claim produces at most one completed delivery; uncertainty triggers reconciliation.
6. A model response cannot authorize itself, fabricate a test pass or release payment.

7. Acceptance binds the agreement and inspected build revision.
8. A structural edit invalidates readiness; the accepted revision must remain protected during settlement.
9. Stale/unavailable service reports prevent new purchases through the designated hooked pool.
10. Existing claims remain obligations when purchases are paused.
11. Reward settlement and refund are mutually exclusive terminal outcomes.
12. A record disappearing from discovery cannot strand an onchain claim or escrow balance.

5. Architecture and dependency model

5.1 Layers

Layer	Modules	Responsibility
Game, experience and access	M01–M02	World authority, player actions, wallet linking and actor authorization
Work marketplace	M03–M05	Agreements, workflow and discoverable records
Agent execution	M06–M08	Model decisions, permission checks and physical construction
Resource economy	M09–M13	Backing, tokens, escrow and trading
Game-to-market connection	M14–M15	Publish conditions and enforce pool rules
Completion and operations	M16–M19	Redemption, tests, evidence and recovery

These are logical boundaries. The MVP can run several modules within one backend process and one plugin; separate modules do not imply separate deployed services.

5.2 Principal dependency paths

- **Commission:** interface → identity → agreement → Arkiv discovery → escrow funding.
- **Construction:** workflow → 0G proposal → policy validation → bot executor → plugin.
- **Resources:** warehouse deposit → KIT issuance → Uniswap purchase → claim → delivery.
- **Hook:** plugin/worker health → authorized publisher → GameStateRegistry → beforeSwap decision.
- **Completion:** protected plot → checker → report/attestation → requester acceptance → reward claims.
- **Recovery:** chain/game observations → operation journal → reconciliation → safe resume.

Every command crossing a process boundary must carry an actor, job/server context and stable operation identifier. External records and model output are treated as data requiring validation.

6. Module specifications

Each module below defines its owned state, allowed inputs, outputs, working sequence, responsibility boundary, dependencies and completion test. Names and operation surfaces are conceptual interfaces, not implemented APIs.

6.1 Game, identity and access

M01 — World server and Paper plugin

Definition. The authoritative adapter for game observations, protected regions and inventory mutations.

Contract of the module	Specification
Owns	Plot membership, item custody, structural revisions, inspection locks and plugin operation receipts.

Contract of the module	Specification
Accepts	Authenticated commands with actor, job, expected revision and operation ID.
Produces	Observed blocks/items, action receipts, revision changes and service-health signals.
Operations	Reserve plot; authorize actor; observe region; lock structure; deliver items; read controls.

Working sequence. Validate the command and permission, schedule game-thread work, check preconditions, journal the operation, perform it and expose a result. Keep model/RPC waits off the game loop. Reject commands when state is uncertain.

Concrete example. A stranger's block edit is denied. An authorized builder places a repeater from the job stock, incrementing the structural revision.

Boundary. Does not choose a builder, calculate token prices, interpret a request or decide payment. It must enforce permissions even if the backend or bot requests an invalid action.

Dependencies. M02 identity, M03 assignments, M04 command bridge, persistent world/plugin storage and a pinned compatible server version.

Done when. Plot isolation and material consumption work, inspection blocks structural changes, and duplicate commands return a recorded result or reconciliation state.

M02 — Player interface and account linking

Definition. The entry point that connects an authenticated game player with wallet-authorized actions.

Contract of the module	Specification
Owns	Player-wallet association, one-time challenges, session authorization and user-facing navigation.
Accepts	Authenticated game session, signed challenge and explicit player actions.

Contract of the module	Specification
Produces	Verified actor context, reviewable agreement/transaction requests and readable status.
Operations	Link wallet; browse/request work; approve terms; review purchase; inspect; accept; claim.

Working sequence. Issue a short-lived challenge bound to server, player, wallet and nonce. Verify the wallet signature and live game session, consume the challenge, then scope later actions to that actor. Signing occurs in the wallet companion.

Concrete example. Player Alex links a wallet, sees Calculator #42 and signs only the agreement displayed for that job.

Boundary. Display names do not prove ownership. The interface cannot mark a submitted transaction confirmed, store player private keys or bypass contract authorization.

Dependencies. M01 game identity, session store, wallet integration and M04/M05/M12/M13 read/write adapters.

Done when. Replayed/expired challenges fail and the player can understand costs, status and next actions without reading protocol logs.

6.2 Jobs, workflow and discovery

M03 — Agreement and team domain

Definition. The rules and data model defining a commission and the contributors entitled to deliver it.

Contract of the module	Specification
Owns	Agreement versions, task assignments, approved responsibilities and required approval set.
Accepts	Supported request, offer, member consent and approved template/plot.

Contract of the module	Specification
Produces	Canonical signed agreement, agreement digest and approved contributor/task configuration.
Required fields	Server/job ID; template/test versions; plot; spending cap; token; recipients/amounts; deadlines; cancellation/refund rules.

Working sequence. Draft terms, validate feasibility, collect approvals to one canonical digest and freeze the funded version. Use a documented canonical encoding, not arbitrary JSON formatting, for the digest. Reserve task regions before parallel work.

Concrete example. A team agrees to 3/2/1 CRED allocations and separate casing/circuit/control tasks. The lead coordinates but cannot take another member's allocation.

Boundary. The first funded implementation keeps recipients and splits immutable. Replacing a funded contributor requires an explicit supported cancellation/successor agreement or separately designed amendment; a backend edit is insufficient.

Dependencies. M02 identity, M05 offers, M08 manifests, M12 escrow and M01 plot permissions.

Done when. Every party sees identical terms, funded fields cannot drift, and two agents cannot hold conflicting work-region leases.

M04 — Workflow orchestrator

Definition. The durable coordinator of job state and external operations.

Contract of the module	Specification
Owns	Operational job state, task queue, leases, expected versions, outbox and retry history.
Accepts	Authorized player actions, confirmed chain events, game receipts and validated task outcomes.
Produces	Next permitted command, persisted transitions and projections for the UI/Arkiv.

Contract of the module	Specification
Operations	Advance job; lease task; queue action; acknowledge receipt; pause; resume; reconcile.

Working sequence. Evaluate transition preconditions, record intent and outbox entry, dispatch the external operation, then record evidence before advancing. Serialize conflicting transitions per job. Retry using the same operation ID; do not treat every retry as a new action.

Concrete example. After a purchase confirms, queue redemption. If the service restarts, find the prior purchase receipt and do not buy another KIT.

Boundary. Does not override the chain's balances or the plugin's inventory. A cached status is not authority to mint, pay or deliver.

Dependencies. All execution adapters, durable relational storage, worker queue and M19 operational monitoring.

Done when. A restart resumes a partially completed job and parallel workers cannot execute the same active task independently.

M05 — Arkiv marketplace adapter

Definition. The shared publication and query layer for independently owned marketplace records.

Contract of the module	Specification
Owns	Record schemas, Arkiv entity references, publishing authorization and renewal schedule.
Accepts	Verified profile/service/offer writes and workflow milestones.
Produces	Search results, independently submitted offers and discoverable completion references.
Record types	Profile; agent registration; service; job listing; offer; agreement reference; progress; completed job.

Working sequence. Publish typed searchable attributes and payload references. Validate owner/signature and domain/server context. Read offers from

independent publishers. Renew active records before expiration and retain funded agreement copies elsewhere.

Concrete example. A circuit specialist publishes a service, discovers an open calculator job and submits their own offer.

Boundary. Does not hold escrow, prove a build works or make a mutable record into an immutable agreement. Search results can be stale; confirm current authorized state before acting.

Dependencies. Arkiv SDK/RPC, publishing funds, M02 wallet ownership, M03 schema and M04 projection/renewal worker.

Done when. Independent writes and discovery work; unauthorized impersonation is rejected; expired discovery data cannot strand funds or delivery claims.

6.3 Decision model and physical agents

M06 — 0G inference adapter and decision model

Definition. The reasoning layer that converts bounded observations into proposed actions.

Contract of the module	Specification
Owns	Prompt/schema versions, selected model/provider configuration, inference budget and decision records.
Accepts	User request or task observation; approved terms; available tools; relevant inventory/test evidence.
Produces	Structured action proposal with job/revision, selected tool, parameters, evidence references and reason.
Decision points	Template selection; task choice; blocked-task response; team scheduling; evidence-based repair.

Working sequence. Build minimal relevant context, invoke the selected 0G-hosted model, parse and validate the response, and hand the proposal to M07. If parsing or inference fails, retry within a bounded policy or wait/escalate. Do not silently replace a required 0G call with another provider.

Concrete example. The model sees six carry-related test failures and proposes inspecting a specific carry-path region before repair.

Boundary. Cannot choose new recipients, approve spending beyond authorization, fabricate observations, declare an inspection passed or accept its own work. Structured-output/tool support must be benchmarked for the selected provider.

Dependencies. 0G access and compute funding, M04 observations, M07 schemas, M08 blueprint and M17 evidence.

Done when. The selected model handles representative valid/invalid requests and failure scenarios; rejected output never reaches execution. Exact model/provider remains an open decision.

M07 — Tool policy and execution authorization

Definition. The deterministic gate between a model proposal and an action with consequences.

Contract of the module	Specification
Owns	Tool definitions, actor capabilities, parameter constraints, execution quotas and authorization decisions.
Accepts	Model or human proposal plus verified job/actor/revision context.
Produces	Authorized command or typed refusal: out of scope, stale, over budget, forbidden or unavailable.
Tool examples	inspect_plot; read_job_inventory; quote_kit; build_stage; compare_blueprint; repair_region; request_inspection.

Working sequence. Validate schema, actor, job stage, revision, plot bounds, task lease, materials and monetary authorization. Attach a stable command ID. Recheck time-sensitive conditions at execution rather than relying only on the earlier model context.

Concrete example. The proposal asks to repair outside the plot. M07 rejects it and records the reason without moving the bot.

Boundary. Neither a fluent explanation nor a high confidence score grants permission. Model instructions cannot weaken the tool policy or hand the bot

administrative credentials.

Dependencies. M02 actor context, M03 terms, M04 leases, M01 permissions and M08/M13 execution adapters.

Done when. Disallowed or stale actions fail consistently regardless of which model, player or retry requested them.

M08 — Blueprint registry and building agent

Definition. The versioned construction knowledge and deterministic executor for the bot.

Contract of the module	Specification
Owns	Blueprint/test-manifest links, material bill, stage checkpoints, navigation and placement routines.
Accepts	Authorized stage/repair command, plot origin/orientation and delivered inventory.
Produces	Observed progress, consumed-item accounting, structural revision and completion/blocked receipt.
Agent identity	Bot ID tied to an operator and approved service/tool permissions; no independent payout identity by default.

Working sequence. Resolve the pinned blueprint, inspect existing blocks, obtain required items and execute a bounded stage. Verify orientation and block state, checkpoint progress and stop on mismatches. Resume by comparing observed reality with checkpoints.

Concrete example. Install two A levers, two B levers and three lamps at the exact approved positions, then connect the known redstone design.

Boundary. Does not invent arbitrary circuits or place unlimited creative-mode blocks to bypass procurement. A bot's completion receipt is not an inspection pass.

Dependencies. Mineflayer and navigation support, compatible game versions, M01 plot/inventory, M07 authorization and a validated blueprint.

Done when. Repeated builds consume the expected stock, produce the intended controls and recover from a disconnect without duplicating completed stages.

6.4 Stock, tokens, contracts and procurement

M09 — Warehouse and backing ledger

Definition. The custody system that reserves real in-game materials for voucher obligations.

Contract of the module	Specification
Owns	Locked deposits, item-by-item reserves, bundle definition and deposit/delivery reconciliation.
Accepts	Supplier deposits and verified delivery receipts.
Produces	Unique issuance authorizations, stock availability and mismatch alerts.
Accounting unit	One versioned KIT bundle; each required item quantity is specified before issuance.

Working sequence. Move deposited materials into controlled custody, record a unique deposit ID and lock the stock before issuance authorization. Count outstanding KIT and undelivered burned claims as obligations. Release reserve obligations only against reconciled delivery.

Concrete example. Ten bundles back ten KIT. Burning two KIT creates two pending claims; their materials stay reserved until delivered.

Boundary. Cannot mint based on a claimed chest balance without locked custody. Token burning does not erase an undelivered obligation. A game snapshot is not independent proof against a dishonest operator.

Dependencies. M01 inventory journal, M11 issuance, M16 claims, M19 reconciler and the finalized material bill.

Done when. A duplicate deposit cannot support a second mint; every required item covers its obligation; mismatches stop new issuance.

M10 — CRED spending-token contract

Definition. The proposed demonstration ERC-20 asset for purchasing kits and paying labor.

Contract of the module	Specification
Owns	Balances, allowances, total supply and declared minting authority.
Accepts	Authorized issuance and standard token transfers/approvals.
Produces	Token movements and events used by the pool and escrow.
Configuration	Settlement chain, contract address, decimals, issuance policy and disclosed initial balances.

Working sequence. Create controlled demonstration supply, distribute initial balances and use consistent base-unit arithmetic throughout wallet, pool and escrow interactions.

Concrete example. A requester authorizes up to 3 CRED for one kit and funds a separate 6 CRED labor escrow.

Boundary. Does not represent KIT stock and does not automatically cover native network gas or 0G inference charges. The document makes no real-value or cash-out promise.

Dependencies. Chosen settlement chain, wallets, M12 token handling and M13 pool liquidity.

Done when. Balances and decimals match across the interface, swaps and reward allocations, with no hidden issuance assumption.

M11 — KIT voucher and issuance contract

Definition. The transferable token claim on the immutable calculator-material bundle.

Contract of the module	Specification
Owns	KIT supply/balances, issuance role, used deposit IDs and bundle identifier.
Accepts	Authorized deposit-based mint request and redemption-authorized burn.
Produces	KIT tokens, issuance events and burn evidence.
Conceptual operations	issueAgainstDeposit(depositId, quantity, recipient); token transfer; authorized redemption burn.

Working sequence. Verify the issuance authorization and unused deposit ID, mint only the authorized amount and mark the deposit consumed in the same transaction. Redemption uses the designated claim contract so a burn and claim creation succeed together or revert together.

Concrete example. A supplier locks three complete bundles and receives three KIT once, then sells KIT into the market.

Boundary. The contract trusts the authorized deposit attestor. It cannot inspect a Minecraft chest. A changed bundle must not silently reuse existing KIT's promise. Whole-kit redemption rejects fractions rather than rounding them up.

Dependencies. M09 locked stock/attestations, token-role administration, M13 trading and M16 redemption.

Done when. Unauthorized/replayed issuance fails; whole-kit burn creates the matching obligation; transferability and fractional-balance treatment are explicit.

M12 — Job escrow and contributor entitlements

Definition. The contract that protects fixed labor allocations and enforces acceptance/refund outcomes.

Contract of the module	Specification
Owns	Funded agreement digest, requester, token, recipients/amounts, deadlines, acceptance and claimed/refunded flags.
Accepts	Required agreement approvals, funding, valid passing-inspection attestation, requester acceptance and contributor claims.
Produces	Funded/accepted/refunded events and one-time contributor entitlements.
Conceptual operations	fundAgreement; registerPass/validatePass; acceptRevision; claimReward; refundExpired.

Working sequence. Verify the approved terms and full funding. Validate server-signed inspection evidence bound to chain, contract, job, agreement, revision and validity. Require requester acceptance before expiry. Atomically choose accepted or refunded; after acceptance each recipient claims a fixed amount.

Concrete example. A 3/2/1 CRED team allocation becomes claimable after acceptance. One failed withdrawal does not block other contributors.

Boundary. Does not judge redstone, custody materials or let the model accept. Structural protection during review is enforced by M01; the contract trusts the inspection attestor. Funded splits are immutable in the first implementation.

Dependencies. M03 signed terms, M10 spending token, M17 attestor, M02 acceptance and M19 confirmed event tracking.

Done when. Duplicate claims fail, acceptance/refund cannot both win, recipients cannot change and all held labor has a defined terminal path.

M13 — Procurement adapter and Uniswap v4 market

Definition. The quoting and execution boundary for exchanging CRED and KIT through the designated pool.

Contract of the module	Specification
Owns	Pool/router configuration, quote expiry, required KIT amount and spending-limit checks.
Accepts	Authorized purchase request with job, quantity, cap and wallet.
Produces	Reviewable quote, transaction request and confirmed swap receipt.
MVP execution choice	Requester signs the bounded purchase; later delegated execution requires a separate limited authorization design.

Working sequence. Quote an exact whole-kit output on the tested route, show expected and maximum spend, and execute with an expiry and enforced cap. Record the transaction and resulting balances. Verify the intended route/asset pair and support hook-aware execution.

Concrete example. One KIT quotes at 2.7 CRED below a 3 CRED cap. If execution needs more than the cap, it reverts instead of consuming labor funds.

Boundary. Swapping is not redemption. Liquidity must actually be seeded. No assumption that a newly created hooked pool appears automatically in a hosted routing API. The adapter cannot draw from labor escrow.

Dependencies. M10/M11 assets, M15 hooked pool, tested router/quote integration, chain RPC and M02 wallet approval.

Done when. A real trade through our pool is confirmed, expired/over-cap trades fail and retrying a submitted transaction does not initiate another purchase.

6.5 Translating game conditions into market rules

M14 — State publisher and GameStateRegistry

Definition. The oracle path for reporting limited game-service conditions on the settlement chain.

Contract of the module	Specification
Owns	Authorized publisher, server/market ID, sequence, submitted time, availability and bounded validity deadline.
Accepts	Authenticated worker/plugin health observations and authorized onchain updates.
Produces	Latest accepted service report and freshness/availability values for the hook.
Conceptual operation	publishState(serverId, marketId, sequence, deliveryAvailable, validUntil).

Working sequence. Observe health, produce a newer sequence and submit the update. The registry checks authorization, context, increasing sequence and bounded lifetime. Publish on changes and refresh before expiry; select heartbeat/freshness settings after chain-cost and outage testing.

Concrete example. If delivery fails, publish unavailable. If the publisher itself dies, the previous healthy report expires.

Boundary. Reports have delay between observation, submission and confirmation. This is not instantaneous world synchronization or independent proof of inventory. Registry publication is separate from voucher issuance and inspection authority.

Dependencies. M01/M16 health, chain RPC/gas, publisher key management and M15 consumer.

Done when. Unauthorized/replayed/overlong reports fail; stale reports are detectable and an old report cannot overwrite a newer state.

M15 — GameStateMarketHook

Definition. The Uniswap v4 pool extension that enforces purchase rules using the reported state.

Contract of the module	Specification
Owns	Expected PoolManager/pool configuration, registry reference and permitted callback behavior.
Accepts	Authorized beforeSwap callback plus pool currencies and swap parameters.
Produces	Allow or revert for a purchase through this pool.
Rule	CRED → KIT requires a fresh, available report for the configured game market.

Working sequence. Verify callback origin and expected pool, determine asset direction correctly and read registry freshness/availability. Reject an unhealthy purchase. Leave the intended sell direction and liquidity removal outside this purchase gate. Test exact-input and exact-output swaps.

Concrete example. A confirmed service-outage report causes the next KIT purchase through this pool to fail with a reason the UI can explain.

Boundary. Does not mint KIT, change warehouse stock, auto-adjust reserves, deliver items, guarantee backing or gate every secondary market. It cannot directly query Minecraft. Existing claims remain enforceable outside the swap path.

Dependencies. M14 registry, v4 deployment, valid hook-address permissions, M13 route and both-direction integration tests.

Done when. Healthy purchases succeed; unavailable/stale ones fail; selling and liquidity removal behave as specified. Hook is attached at pool creation, with only required permissions.

6.6 Delivery, verification and operations

M16 — Redemption claim and delivery worker

Definition. The recoverable path from burned KIT to job inventory.

Contract of the module	Specification
Owns	Onchain claim identity/target/quantity and durable worker/plugin delivery records.
Accepts	Authorized whole-kit redemption, confirmed claim event and plugin receipt.
Produces	Burn-and-claim transaction, pending delivery, completed receipt and optional server-attested chain acknowledgement.
Claim binding	Server, job, recipient, bundle version and quantity are fixed or authorized at creation.

Working sequence. Burn and create the claim atomically onchain. After the chosen confirmation policy, reserve the worker operation by claim ID, request plugin delivery and reconcile its durable receipt. If acknowledgement is stored onchain, post it only after the game receipt is established.

Concrete example. A crash occurs after items enter the inventory but before the worker records success. Recovery checks the plugin receipt before attempting any further delivery.

Boundary. An event listener is not sufficient duplicate protection. Game mutation and database writes need a tested journal/reconciliation design. Acknowledgement attests delivery; it does not itself deliver or prove honest custody.

Dependencies. M09 reserves, M11 burn authority, M01 protected inventory, M04 queue and M19 confirmation/reorganization policy.

Done when. Offline/full destinations remain pending, replay cannot duplicate delivery and ambiguous outcomes are quarantined for reconciliation.

M17 — Functional checker and inspection attestor

Definition. The deterministic verifier of calculator behavior and producer of revision-bound pass evidence.

Contract of the module	Specification
Owns	Pinned test manifest, reset/timing procedure, test outcomes and inspection-signing authority.
Accepts	Authorized submission for a locked job plot and structural revision.
Produces	16-case report, failed-case evidence and a bounded pass attestation only on success.
Interface	A and B: 0–3 using two weighted levers each; output: three lamps weighted 1, 2, 4.

Working sequence. Acquire structural lock, reset, set each input pair, wait under validated timing, read lamp block states and compare with A+B. Stop/invalidate if a structural mutation occurs. On pass, retain protection through review/acceptance or expiry.

Concrete example. A=3 and B=1 must show 4. A permanently absent high carry output fails the six sums of 4 or more.

Boundary. Does not use screenshots or model judgments as functional truth. Interacting with approved input levers is allowed during review; structural edits are not. Attestation remains trusted server evidence.

Dependencies. M01 lock/revision, M08 interface manifest, M18 evidence store and M12 signed-evidence validation.

Done when. Correct 16/16, fault 10/16 and repaired 16/16 are repeatable; a changed or expired revision cannot reuse the old pass.

M18 — Reports, progress and completion history

Definition. The evidence presentation and publication layer.

Contract of the module	Specification
Owns	Durable report/artifact references, readable milestone projections and completion-record formatting.
Accepts	Verified workflow events, test results and confirmed settlement evidence.

Contract of the module	Specification
Produces	In-game status, companion detail, inspection case table and Arkiv completion reference.
Publication rule	Only authorized evidence changes the visible lifecycle; secrets and private sessions are excluded.

Working sequence. Retain the full report and agreement copy, derive player messages from events, link hashes to retrievable artifacts and publish completion references. Explain pending versus confirmed states.

Concrete example. The UI changes from 10/16 repair required to 16/16 ready for review, then accepted with each contributor's claim status.

Boundary. Does not invent success or become a new source of payment authority. A hash identifies content, not the truth of its world observations. Arkiv expiry must not erase the only report copy.

Dependencies. M04 events, M05 discovery, M17 reports, M12/M16 receipts and durable artifact storage.

Done when. Every displayed success links to supporting evidence, and participants can recover pending claim/agreement details independently of marketplace availability.

M19 — Recovery and administration

Definition. The operational layer for partial failures, reconciliation and authorized maintenance.

Contract of the module	Specification
Owns	Backups, incident state, reconciliation jobs, health alerts and key/role lifecycle.
Accepts	Chain/game observations, failed operations and authenticated operator actions.
Produces	Safe resume, paused operations, mismatch reports and auditable repairs.

Contract of the module	Specification
Key separation	Publisher, issuer, inspection attestor and administrator use distinct roles; no bot/player key reuse.

Working sequence. Persist operation intents and receipts, index confirmed chain events with reorganization handling, compare stock and obligations and back up game/database state under a recovery plan. Reconcile before replaying uncertain work.

Concrete example. The backend sees a completed swap after restart and resumes the existing claim, while a separate inventory mismatch stops issuance for operator review.

Boundary. Administration cannot silently substitute payout recipients, forge requester acceptance or resolve an undefined ownership policy by editing balances. Recovery repairs evidence/state under documented authority.

Dependencies. Durable storage, world/plugin journal, all contract events, access-controlled operator interface and deployment manifest.

Done when. Restart, duplicate-event, stale-state and partial-delivery drills preserve obligations and expose anything unresolved.

7. Data and interface contracts

7.1 Core records

These are logical data contracts. Final storage encoding, contract ABI and SDK types are implementation outputs.

Record	Required content	Ownership / retention
PlayerProfile	Player ID, verified wallet association, display name, supported services and agent references	Association verified by M02; public subset in Arkiv
Job	Server/job ID, requester, active agreement version, plot, workflow state, revision and timestamps	M04 operational record; searchable projection in Arkiv

Record	Required content	Ownership / retention
Agreement	Canonical terms, parties/approvals, template/test versions, exact allocations, cap, deadlines, acceptance/refund policy and digest	Retained signed copy; digest and critical settlement fields onchain
BlueprintManifest	Version, material bill, orientation, stages, interface coordinates and linked test manifest	Versioned registry; immutable for a funded agreement
AgentDecision	Job/task/revision, model/provider, prompt/schema version, observation references, proposal and policy outcome	Private operational audit record; redact secrets
Deposit	Unique ID, supplier, locked quantities, bundle version, issuance authorization and mint transaction	Warehouse journal plus used-ID protection onchain
DeliveryClaim	Chain/contract/claim ID, job/server/recipient, quantity, confirmation state and delivery receipt	Onchain creation plus durable worker/plugin records
InspectionReport	Job/agreement/revision/test version, each input/expected/observed result, timing, report digest and attestation validity	Durable full evidence; public reference as appropriate
RewardAllocation	Agreement, contributor, exact amount, accepted/refunded/claimed state and transaction references	Escrow source of truth; UI projection
GameServiceReport	Server/market, sequence, delivery availability, submitted time and validity deadline	Registry state from authorized publisher

Amounts use explicit token address, chain and base-unit quantities. Identity fields must not rely on display names. The same job ID is namespaced by server/application so records cannot be confused across deployments.

7.2 Cross-module command envelope

Every asynchronous command must identify:

- **Command/operation ID:** stable across retries of the same intended action.
- **Context:** server, job, agreement version and expected build/task revision.

- **Actor:** verified player, bot operator or service role.
- **Action and parameters:** typed, bounded and validated.
- **Authorization reference:** approval, signed request, task lease or contract event.
- **Expiry and preconditions:** no execution against stale assumptions.
- **Result:** success with evidence, retryable failure, permanent refusal or reconciliation required.

A model decision ID is not automatically an operation ID. One proposal may be rejected, or may authorize a separate command after validation.

7.3 Event and transaction handling

Distinguish requested, submitted, included/observed, confirmed, applied and reconciled states. A submitted transaction may fail or be replaced. A confirmation policy must be chosen for the settlement chain before game delivery.

Event consumers deduplicate by chain, contract, transaction and log identity, while also tracking semantic IDs such as deposit or claim. Reorganization handling must revisit observations; it must not blindly replay irreversible game effects.

A game mutation uses a plugin receipt and reconciliation protocol. A crash between a world change and a database commit cannot be assumed harmless merely because the message has an ID.

7.4 Dependency availability

Missing dependency	Operations that must wait	Operations that can remain available
0G inference	New model-dependent task selection	Read status; complete already authorized deterministic work if safe
Arkiv	New publication/discovery and renewal	Existing signed agreements, escrow claims and retained job views
Settlement RPC	Purchases, confirmed claim processing and settlement	World exploration; already delivered-material construction

Missing dependency	Operations that must wait	Operations that can remain available
	confirmation	if terms allow
Delivery plugin/worker	Delivery and new purchases once registry state reflects outage or expires	View claims; preserve obligations; permitted token sales
Checker	Readiness/pass attestation	Construction and repair within authorized scope
Publisher	New purchases after report expiry	Redemption claims remain valid; delivery may continue if healthy

A state update takes time to become effective onchain. The hook responds to its latest confirmed readable state, so the design must not promise instant outage detection.

8. End-to-end operating flows

8.1 Flow A — Request, discovery and agreement

Step	Actor / module	Action and resulting artifact	Gate
1	Requester, M02	Link authenticated game account and wallet	Valid one-time challenge
2	Requester, M06	Describe calculator; receive supported template proposal	Unsupported requests cannot become arbitrary build promises
3	M03	Draft requirements, materials cap, tests and deadlines	Template and plot feasible
4	M05	Publish job; discover independent builder offer	Owner/signature checks
5	Requester and builder/team	Approve identical agreement digest	All required approvals collected

Step	Actor / module	Action and resulting artifact	Gate
6	Requester, M12	Fund fixed labor allocations	Onchain agreement fields match approved terms

For teams, the model may suggest assignments, but humans approve responsibility and reward before funding. Do not estimate “fair contribution” from block counts and silently redistribute rewards.

8.2 Flow B — Backing, trade and delivery

Supplier preparation happens before the requester purchase: lock real materials → unique authorized issuance → KIT available for sale/liquidity.

Step	Module	Operation	Completion evidence
1	M13	Quote one whole KIT under the cap	Quote with route, expiry and maximum input
2	M02	Requester signs bounded purchase	Wallet authorization
3	M15 → M14	Check configured pool and reported availability/freshness	Hook allows swap or reverts
4	M13	Confirm the actual CRED/KIT exchange	Receipt and expected balance change
5	M16 → M11	Burn KIT and create job-bound claim	Atomic onchain burn/claim evidence
6	M16 → M01	Deliver backed materials using claim ID	Durable inventory receipt
7	M09/M04	Reconcile reserves and advance to construction	Ledger and workflow evidence agree

If an over-cap trade reverts, no purchase is recorded as complete. If delivery fails after burning, the claim persists. If the receipt is uncertain, reconciliation precedes retry.

8.3 Flow C — Model decision to real construction

1. M04 leases an eligible task and records the relevant revision.
2. M01/M08 provide actual plot, stock and checkpoint observations.
3. M06 asks the selected 0G model for one bounded next action.
4. M07 validates actor, stage, materials, region, revision and tool limits.
5. M08 executes the authorized stage through the bot.
6. M01 reports observed results and structural changes.
7. M04 checkpoints progress; M18 updates the player.
8. On a blocked task, M06 may propose another permitted task or escalation.

Example: the casing is complete and a teammate owns the circuit region. The model chooses installing input controls in a separate approved region. No inference call is required for each individual block.

8.4 Flow D — Inspection, repair and acceptance

1. Builder submits the current structural revision.
2. Checker acquires a structural lock and pins agreement/test versions.
3. For each A/B pair, reset, set inputs, wait and read output lamps.
4. Retain all expected/observed results and supporting screenshots.
5. On failure, return to repair; no pass attestation or payout is created.
6. The model uses failed cases plus actual blueprint differences to propose a repair.
7. Policy validates and bot performs it; repeat all 16 tests.
8. On success, issue a revision-bound pass with validity.
9. Keep structural edits locked while the requester tries the controls and reviews.
10. Requester accepts the exact inspected revision before expiry.
11. Escrow atomically records acceptance; contributors claim fixed rewards.
12. Publish completion and keep the accepted structure under the declared ownership/permission policy.

The lock prevents the window between inspection and acceptance from being used to alter the structure. An early edit/unlock invalidates readiness and requires a new pass.

8.5 Flow E — Game-state change to hook enforcement

1. Delivery health changes or a refresh is due.
2. Publisher observes the condition and submits a newer report.
3. Registry validates the publisher, context, sequence and bounded lifetime.
4. After inclusion, later pool purchases read the updated state.
5. An unavailable or expired report causes a CRED-to-KIT purchase to revert.
6. After a healthy fresh report, a later purchase can succeed under normal quote rules.

There is no direct chain call to inspect the Minecraft server. Freshness bounds how long an old report can remain acceptable; it does not remove the need to trust the reporter.

8.6 Flow F — Teams and departures

The whole job has one acceptance result. Tasks have individual assignees and work-region leases. The lead submits the result, but labor entitlements belong directly to the named contributors.

For the MVP, a funded participant's reward is not reallocated automatically if they disconnect or leave. Pause their tasks, recover their permitted stock and follow the agreement's cancellation/successor procedure. Do not put an unsupported amendment button in the interface.

Separately accepted milestones, proportional partial payment and automatic contribution scoring require a later policy/contract design.

9. Lifecycle, money and failure semantics

9.1 Job transitions

State	Entry evidence	Exit authority / next condition
Draft	Supported request and editable job card	Requester publishes
Open	Discoverable listing	Requester selects an approved offer
Agreed	Required signatures on one digest	Requester funds
Funded	Confirmed labor funding	Authorized procurement begins
Procuring	Recorded purchase operation	Confirmed KIT acquisition
Delivery pending	Confirmed redemption claim	Reconciled inventory receipt
Building	Delivered resources and permitted task	Builder submits
Inspecting	Locked revision and test run	Checker passes or fails
Repair required	Actual failed-case report	Builder resumes permitted repair
Ready for review	Valid pass for protected revision	Requester accepts or review expires
Accepted	Confirmed authorized acceptance	Contributors claim allocations
Paid	All reward allocations settled	Terminal
Cancelled / Refunded	Contract-permitted cancellation/timeout outcome	Terminal; resource obligations tracked separately

Temporary waiting reasons such as unavailable inference, stale service report or blocked inventory are substates, not permission to skip required transitions.

9.2 Budget and custody

Illustrative budget: 10 CRED total, comprising a 3 CRED materials cap, 6 CRED labor and 1 CRED unallocated reserve. The requester keeps funds not explicitly transferred/authorized.

Category	Custody / treatment
Labor reward	Escrowed in CRED; fixed allocations; accepted or refundable under terms
Unspent materials authorization	Still requester funds in the MVP; cap is not a charge

Category	Custody / treatment
Purchased unredeemed KIT	Owned by the designated purchase recipient; can be redeemed or traded under market conditions
Burned KIT with pending claim	Delivery obligation; not automatically converted back to CRED
Delivered unused items	Requester/job-owned under proposed policy; warehouse redemption promise already fulfilled
Placed/consumed materials	Part of the work; cancellation does not reverse construction or a market trade
Gas/inference charges	Separately disclosed network/service expenses

Proposed resource-ownership policy: purchased KIT, delivered materials and the commissioned structure belong to the requester, with builder/team access limited to performing the job. Unused stock returns to requester-controlled access when the job terminates. This policy needs explicit agreement and plugin enforcement before use.

9.3 Deadlines and cancellation

The agreement must specify a work deadline, a review window and an absolute escrow expiry. Resubmission cannot extend the absolute expiry without an explicitly supported agreement change.

Proposed MVP policy: before acceptance, a contract-defined expired job makes held labor refundable to the requester. Acceptance before expiry makes allocations claimable and prevents refund. A contributor's unclaimed accepted allocation remains theirs; it must not become a requester refund merely because they withdraw late.

Before funding, draft/open jobs may be withdrawn. After funding, only the contract's supported mutually approved cancellation or timeout route applies. Purchased goods and pending claims retain their separate ownership/obligations.

This policy is simple but can leave an honest builder unpaid if the requester refuses acceptance. Do not claim fair dispute resolution or guaranteed earned payment. A later arbitration/milestone policy would change the product and requires separate design.

9.4 Failure-response matrix

Failure	Immediate action	Recovery condition
Invalid model output	Reject; log; bounded retry or wait	Valid permitted proposal
Task revision changed	Reject stale command	New observations and authorization
Quote exceeds cap	Do not submit / revert under limit	New valid quote or explicit revised authorization
Hook reports unavailable/stale	Reject new purchase through our pool	Confirmed healthy fresh report
Arkiv record expires	Republish/renew under policy	Verified restored discovery; existing contracts unaffected
Claim observed, chain uncertain	Wait/reconcile	Chosen confirmation/reorganization policy satisfied
Delivery destination offline/full	Keep claim pending	Available protected inventory
World mutation/receipt ambiguous	Quarantine operation	Plugin journal plus inventory reconciliation
Bot disconnects	Release/expire safe task lease; pause	Reconnect and inspect checkpoint
Inspection fails	Keep labor held; record failures	Repair and complete rerun
Structure changes in review	Invalidate readiness	New locked revision and passing report
Acceptance races refund	Contract chooses one terminal result	Confirm and reflect the actual chain outcome
Stock falls below obligations	Stop new issuance; investigate	Item-level reconciliation and authorized remediation

10. Delivery plan and definition of done

10.1 Dependency-led build order

Stage	Deliverable	Gate before moving on
A — Functional world	Plot permissions, known calculator and deterministic checker	Repeatable correct/fault/repaired outcomes
B — Physical agent	Bot constructs approved stages using delivered stock	Material accounting and resumable construction
C — Decision layer	Real 0G request, structured proposals and tool policy	A meaningful decision changes allowed work
D — Agreements/discovery	Identity, signed job and independent Arkiv offers	Two real participants and ownership checks
E — Resource economy	Locked deposits, issuance, pool trade and redemption	Actual backed purchase and duplicate-safe delivery
F — State-aware pool	Publisher, registry and beforeSwap hook	Healthy/stale/unavailable tests on the real route
G — Settlement	Fund, inspect, accept, claim and refund	Fixed allocations; no duplicate or contradictory settlement
H — Complete experience	Menus, companion, reports and recovery drills	One coherent end-to-end demonstration

These are validation dependencies, not committed estimates or separate agent assignments. The team can implement independent modules concurrently once shared interfaces are agreed.

10.2 Acceptance checklist for the finished MVP

- [ ] Requester can understand and approve the supported calculator job.
- [ ] Independent builder publishes an offer and is selected through Arkiv.
- [ ] Wallet/game linkage and agreement approvals reject replay/impersonation.
- [ ] Actual 0G inference produces a recorded, permitted action.
- [ ] A backed KIT is issued from locked stock and traded through our pool.
- [ ] Unavailable/stale game-service state blocks a purchase through the hook.
- [ ] A confirmed claim delivers materials without duplicate fulfillment.
- [ ] The bot visibly builds using those resources in the assigned plot.

- [ ] The disclosed carry fault yields 10/16; repair yields 16/16.
- [ ] The inspected structure stays protected through review and acceptance.
- [ ] Requester acceptance enables fixed reward claims once.
- [ ] Timeout/cancellation and purchased-material ownership are explicit.
- [ ] A restart during purchase/delivery resumes without repeating the charge or delivery.
- [ ] Completion includes the working in-world structure, report and settlement references.
- [ ] Prepared fixtures, seeded liquidity and deliberately introduced faults are disclosed.

10.3 Decisions still required

Decision	Current framing	What resolves it
Server/bot versions	Pin one compatible combination	Construction and timing tests
Final blueprint/material bill	Fixed two-input calculator	Repeated successful construction/inspection
0G model/provider	Decision model through 0G; exact selection open	Valid-action benchmark, latency/cost and applicable funding requirements
Settlement chain	One v4-supported test environment; Sepolia is a candidate	Verify deployments, route, wallet and available funds
Registry freshness settings	Bounded TTL and refresh interval	Cost/outage-latency tests; declared tolerated delay
KIT decimals/fractions	Whole-kit redemption; fractions remain tradable	Token/quote integration and clear UX
Inspection evidence format	Signed pass bound to job/agreement/revision	Contract/interface tests and lock lifecycle
Team scope	Schema supports fixed allocations; one builder first	Complete single-builder flow before team demonstration

Decision	Current framing	What resolves it
Refund/ownership policy	Proposed rules in section 9	Explicit adoption in agreement and contract
Platform suitability	Public token-linked deployment needs review	Current Minecraft rules; compatible voxel platform if required
Hackathon eligibility	Track and pre-event work remain external constraints	Current official requirements and organizer guidance

10.4 What this specification does and does not establish

The document defines a target experience, module boundaries, interface obligations and acceptance criteria. It does not establish a working server, tested contract security, benchmarked model, qualifying sponsor submission or permission for a public token economy.

The initial server operator remains trusted for world state, inventory custody, inspection and service reports. Decentralized records and contracts make selected commitments inspectable and enforceable; they do not independently prove that a reported world observation is truthful.

11. Technical basis and references

The proposed AgentCity architecture is our design. These official sources support the underlying capabilities:

- **Paper:** plugin development foundation for custom server behavior. Paper development documentation
- **Mineflayer:** bot control APIs used as the construction foundation. Mineflayer API
- **Arkiv:** queryable entities, ownership and expiration; application contracts must be deployed elsewhere. Arkiv fundamentals
- **0G:** model-inference access through its compute interfaces; AgentCity supplies prompts, tools and permissions. 0G Compute

- **Uniswap v4:** optional pool hooks around operations such as swaps, configured at pool creation. Hook concepts
- **Hook implementation reference:** deployment and callback examples; not AgentCity's completed implementation. Building a first hook
- **Platform review:** consult current terms before the intended deployment. Minecraft usage guidelines